

TODO

TODO

1 de junho de 2026

Resumo

1 Introdução

Os sistemas distribuídos modernos estão sujeitos a requisitos rigorosos de disponibilidade e baixa latência que, na presença de partições de rede, impossibilitam a garantia de consistência forte. O compromisso estabelecido pelo Teorema CAP obriga a adoção de modelos de consistência mais relaxados, como a Strong Eventual Consistency (SEC). Neste contexto, os Conflict Free Replicated Data Types (CRDTs) assumem um papel central, permitindo que diferentes réplicas sejam atualizadas de forma concorrente e sem necessidade de coordenação centralizada, garantindo matematicamente a convergência do estado.

Apesar destas propriedades teóricas, o design e implementação corretos de CRDTs em aplicações reais continuam a ser um desafio. Basta uma falha na lógica da função de combinação de estado para resultar na divergência permanente ou ir contra a intenção do utilizador. Para mitigar estes problemas, o recurso à verificação formal tem-se revelado uma necessidade, dividindo-se o estado da arte em duas abordagens principais: a verificação automática e a verificação dedutiva.

Ferramentas automatizadas com recurso a SMT solvers, como a linguagem VeriF_x, são altamente eficazes na validação rápida de propriedades algébricas de convergência. No entanto, esbarram em limites de expressividade quando necessitam de raciocinar sobre estruturas recursivas não limitadas ou sobre invariantes de estado globais. Por outro lado, plataformas de verificação dedutiva como o Why3 oferecem o poder expressivo necessário para estabelecer semânticas correct by construction, recorrendo a contratos explícitos e ghost code. A grande desvantagem desta abordagem reside no

esforço manual exigido e na exigente aprendizagem imposta aos programadores.

A motivação do trabalho aqui apresentado surge da necessidade de preencher esta lacuna metodológica. Propomos uma framework integrada para o desenvolvimento de CRDTs, com o objetivo de aliar a conveniência da verificação automática à robustez das provas dedutivas. Para concretizar esta integração, desenhou-se uma nova linguagem de domínio específico que captura a semântica dos CRDTs.

Neste artigo, focamo-nos na apresentação e implementação do pipeline de compilação da nossa framework. O sistema processa o código fonte através de análise lexical e sintática, garante a sua correção através de um sistema de tipos e, por fim, traduz a Árvore de Sintaxe Abstrata (AST) num programa WhyML equivalente, pronto a ser submetido à plataforma Why3 para prova formal. Esta estratégia de tradução tem como principal contribuição a ocultação da complexidade inerente à verificação dedutiva, reduzindo drasticamente o esforço manual de especificação exigido ao programador, ao mesmo tempo que mantém garantias matemáticas sobre a convergência e a integridade.

2 Verificação Formal de CRDTs

3 Breve Descrição da Arquitetura da Framework

4 Pipeline de Compilação para Verificação Formal

5 Casos de Estudo

6 Trabalho Relacionado

7 Conclusão e Trabalho Futuro